UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants. Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2014-06, page [3]

Unofficial model answer:

The complete machine cycle consists of the instruction cycle (fetch and execute phases) and, when required, the interrupt cycle. In the fetch phase, the CPU transfers the address in the program counter (PC) to the memory address register (MAR), issues a memory read, and loads the instruction from memory into the memory data register (MDR), then into the instruction register (IR); the PC is incremented. The instruction is decoded to determine the operation and operand addressing. In the execute phase, the CPU performs the required operation: it may fetch operands from memory or registers, perform arithmetic/logic operations in the ALU, and write results back to registers or memory. After execution, the CPU checks for pending interrupts; if an interrupt is present and enabled, it enters the interrupt cycle: it saves the current PC (and processor state) on the stack or in a saved-state register, loads the interrupt service routine address into the PC, and then returns to the fetch phase. If no interrupt is pending, the CPU proceeds directly to the next fetch. The cycle is therefore: Fetch → Decode → Execute → Interrupt check → (Interrupt cycle if needed) → repeat.

Marking points:

- *1 mark: Correctly shows the fetch phase with PC → MAR, memory read, MDR → IR, and PC increment.*
- *1 mark: Correctly shows instruction decode and operand fetch/execute phase.*
- *1 mark: Correctly shows the interrupt check and interrupt cycle (save PC/state, load ISR address) when an interrupt is pending.*
- *1 mark: Clear, labelled diagram with directional arrows showing the sequence and decision point for interrupts.*
- *1 mark: Overall accuracy and completeness of the machine cycle (fetch, indirect (if applicable), execute, interrupt) with correct terminology.*

Model confidence: medium.

Model note: The question asks for a diagram; the answer describes the required diagram elements. Some machine cycle descriptions include an indirect cycle for memory-indirect addressing; this is optional depending on the course emphasis. The marking scheme is an unofficial model answer, not an official UTM scheme.

QUESTION 2 (5 marks)

Source: CSC429-2025-07, page [2]

Unofficial model answer:

a) Inventors: i) Mechanical Calculating Machine — Blaise Pascal (the Pascaline, 1642). ii) Analytical Machine (Analytical Engine) — Charles Babbage (1837). iii) IAS Machine — John von Neumann (Institute for Advanced Study machine, 1945/1952). b) Later-generation machines are general-purpose because they use stored-program architecture (von Neumann model): instructions and data reside in the same rewritable memory, so the same hardware can execute different programs without rewiring. Earlier machines were special-purpose (e.g., Pascal's adder, Babbage's engine, ENIAC's plugboard) with fixed or hard-wired functions. Advances in electronics (transistors, ICs, VLSI) made processors fast, small, cheap and reliable enough to support flexible instruction sets, operating systems and high-level languages, giving versatility, programmability, and better cost-performance across many applications.

Marking points:

- *1 mark: i) Blaise Pascal*
- *1 mark: ii) Charles Babbage*
- *1 mark: iii) John von Neumann*
- *1 mark: General-purpose due to stored-program concept / programmability (instructions and data in same memory,*

© Hak Cipta Universiti Teknologi MARA

no rewiring)

• 1 mark: Contrast with earlier special-purpose/fixed-function machines and/or role of transistor/IC/VLSI advances enabling flexibility, reliability and cost-effectiveness

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. 'Mechanical Calculating Machine' is interpreted as Pascal's Pascaline; if the paper intended Leibniz's stepped reckoner, accept Gottfried Wilhelm Leibniz. 'Analytical Machine' is Babbage's Analytical Engine. IAS machine is attributed to John von Neumann (with Goldstine and Burks).

QUESTION 3 (5 marks)

Source: CSC429-2026-02, page [3]

Unofficial model answer:

The equation is interpreted as computing (-56 decimal) + (A3 hexadecimal). A3H = 10×16 + 3 = 163 decimal. The sum is -56 + 163 = 107 decimal. Using 8-bit two's complement arithmetic: 56 decimal = 00111000_2. 1's complement of 56 = 11000111_2. 2's complement of 56 = 11001000_2 (C8H). A3H = 10100011_2. 1's complement method: add the 1's complement of 56 to A3H: 11000111 + 10100011 = 1 01101010. Since there is a carry out of the sign bit, perform end-around carry: 01101010 + 1 = 01101011_2 = 6BH = 107 decimal. 2's complement method: add the 2's complement of 56 to A3H: 11001000 + 10100011 = 1 01101011. Discard the carry out of the sign bit; the result is 01101011_2 = 6BH = 107 decimal. Both methods give the same positive result, 107 decimal (6BH), which is correct because the magnitude of the positive operand (163) exceeds the magnitude of the negative operand (56).

Marking points:

• 1 mark: Correctly convert -56 to 8-bit 1's complement (11000111) and 2's complement (11001000), and A3H to binary (10100011).

• 1 mark: Perform 1's complement addition: 11000111 + 10100011 = 1 01101010, showing the carry out.

• 1 mark: Apply end-around carry in 1's complement: 01101010 + 1 = 01101011, and interpret as +107 (6BH).

• 1 mark: Perform 2's complement addition: 11001000 + 10100011 = 1 01101011, discard carry, result 01101011 = +107 (6BH).

• 1 mark: State final answer clearly (107 decimal or 6BH) and note that both methods agree and the result is positive.

Model confidence: high.

Model note: Assumes 8-bit word length and that the equation means (-56) + A3H. The original question text '-56s + A3H' is interpreted as '-56 decimal plus A3 hexadecimal'; the 's' is likely a typo or subscript. No figure or table was provided.

QUESTION 4 (5 marks)

Source: CSC429-2025-02, page [2]

Unofficial model answer:

a) The PC was first introduced in the fourth generation of computers. The fourth generation (roughly 1971 onward) is characterised by microprocessors — VLSI circuits with a complete CPU on a single chip — which made it economically feasible to build small, low-cost computers for individual use; the IBM PC (1981) is the classic example of this generation's personal computer.

b) The first commercially available computer was the UNIVAC I (UNIVersal Automatic Computer I), delivered in 1951 and manufactured by Remington Rand. (Note: ENIAC, 1946, was an earlier electronic computer but was not a commercial product; UNIVAC I is the accepted answer for 'first commercially available'.)

c) When the PC was first introduced, the interface available was a command-line interface (CLI). Early PCs such as the IBM PC ran operating systems like MS-DOS/PC-DOS, in which the user typed text commands at a prompt; the graphical user interface (GUI) with windows, icons and a mouse became common only later (e.g., with Apple Macintosh and Microsoft Windows).

d) The key factor that makes a personal computer portable is its compact, integrated hardware built from

© Hak Cipta Universiti Teknologi MARA

microelectronics — specifically the microprocessor (CPU on a single chip) together with VLSI memory and support chips, which drastically reduced size, power consumption and cost. This miniaturisation, combined with lightweight construction and battery power in laptops/portables, allows the whole system to be carried and used by an individual. (If the intended answer is the enabling technology, state 'microprocessor/VLSI integration'; if the intended answer is the design feature, state 'small size, low power and lightweight/battery-powered construction'.)

Marking points:

• 1 mark (a): Fourth generation (microprocessor-based generation).

• 1 mark (b): UNIVAC I (accept UNIVAC; do not accept ENIAC alone).

• 1 mark (c): Command-line interface (CLI) / text-based interface (accept DOS-type command prompt; GUI alone is incorrect).

• 2 marks (d): 1 mark for identifying the key factor (microprocessor/VLSI miniaturisation, or compact low-power lightweight battery-powered design); 1 mark for a brief explanation of how it enables portability.

Model confidence: medium.

Model note: Unofficial model answer, not an official UTM marking scheme. Part (d) is open to two readings (enabling technology vs. physical design feature); both are given. Part (b) depends on the lecturer's convention: UNIVAC I is standard, but some texts cite the Ferranti Mark 1 or EDSAC as 'first commercially available/sold' machines — ENIAC is not commercial. Part (c) assumes the IBM PC/MS-DOS era; if the lecturer dates 'PC' to earlier hobby machines, CLI still applies.

QUESTION 5 (10 marks)

Source: CSC429-2018-01, page [4, 5]

Unofficial model answer:

a) Ranking by price (cheapest to most expensive per bit): tape back-up, hard drive, cache, register. Tape is the cheapest per bit (slowest, offline/sequential backup medium), followed by the hard drive (magnetic disk, cheaper per bit than semiconductor memory), then cache (fast SRAM, expensive per bit), and finally registers (fastest, most expensive per bit, built into the CPU). This ordering follows the memory hierarchy: as speed increases, cost per bit increases and capacity decreases.

b) One benefit of cache memory: it reduces the average memory access time (and hence improves overall system performance) by exploiting locality of reference. Because cache is small, fast SRAM located close to the CPU, frequently used instructions and data (temporal and spatial locality) are kept there, so most accesses are satisfied from cache rather than the slower main memory, reducing the effective memory latency and the number of main-memory accesses.

c) Write through vs write back: In write through, every write by the CPU is written to both the cache block and the corresponding main-memory location simultaneously, so main memory is always kept consistent (up to date) with the cache; this is simple but generates more main-memory traffic. In write back, the write is made only to the cache block, and the main-memory copy is updated only when that dirty block is later replaced/evicted from the cache (a dirty bit marks modified blocks); this reduces main-memory write traffic but requires a dirty bit and makes main memory temporarily inconsistent with cache, and coherence must be managed.

Marking points:

• a) 4 marks: 1 mark each for correct position of tape back-up (cheapest), hard drive, cache, register (most expensive); full 4 marks only if the whole order is correct.

• b) 2 marks: 1 mark for stating the benefit (e.g. reduces average memory access time / improves performance / exploits locality), 1 mark for a correct explanation of how it is achieved.

• c) 4 marks: 2 marks for correct description of write through (write to both cache and main memory, memory always consistent, more traffic), 2 marks for correct description of write back (write only to cache, update memory on eviction using dirty bit, less traffic but possible inconsistency).

• Total: 10 marks.

© Hak Cipta Universiti Teknologi MARA

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. Part (a) assumes price per bit as the standard interpretation of 'price' in the memory hierarchy; if the question intended total cost of the device, the ranking could differ, but the hierarchy-based per-bit interpretation is standard for CSC429.

QUESTION 6 (10 marks)

Source: CSC429-2026-02, page [5]

Unofficial model answer:

Part (a) — Completing the taxonomy (Figure 4). The taxonomy follows Flynn's classification at the top and then subdivides the MIMD branch by memory organization. The missing labels are: **(a) Single Instruction, Single Data (SISD)** — the left branch under 'Single instruction' (a single processor executing one instruction stream on one data stream). **(b) Uniprocessor** — the SISD leaf, i.e. a conventional single-processor machine. **(c) Shared Memory** — the MIMD branch that is subdivided into tightly coupled multiprocessors sharing memory. **(d) Multiple Instruction, Multiple Data (MIMD)** — the right branch under 'Multiple instruction'. **(e) Distributed Memory** — the loosely coupled MIMD branch (each processor has its own private memory, e.g. multicomputers/clusters). **(f) Symmetric Multiprocessor (SMP)** — the shared-memory leaf in which all processors have equal access to a single shared main memory. **(g) NUMA (Non-Uniform Memory Access)** — the other shared-memory leaf, where memory access time depends on which processor/memory module is accessed. So the completed tree reads: Processor Organization → SISD (Uniprocessor) and MIMD; MIMD → Shared Memory (SMP, NUMA) and Distributed Memory.

Part (b) — Two disadvantages of SMP. First, **memory contention / limited scalability**: because all processors share one main memory and a common bus or interconnect, simultaneous accesses contend for the same bandwidth, so adding processors yields diminishing returns and the system saturates (the bus/memory becomes the bottleneck). Second, **cache coherence overhead and synchronization cost**: with private caches per processor, hardware must keep cached copies consistent (snooping or directory protocols), which adds traffic and complexity, and shared-data programs require locking/synchronization that serializes execution and limits speed-up. (Acceptable alternatives: single point of failure in shared memory/bus, higher cost/complexity of coherent hardware, and difficulty of writing correct parallel software.)

Marking points:

- *1 mark: (a) SISD correctly placed under Single instruction*
- *1 mark: (b) Uniprocessor correctly placed as the SISD leaf*
- *1 mark: (d) MIMD correctly placed under Multiple instruction*
- *1 mark: (c) Shared Memory correctly placed as the MIMD subdivision*
- *1 mark: (e) Distributed Memory correctly placed as the loosely coupled MIMD branch*
- *1 mark: (f) Symmetric (SMP) correctly placed under Shared Memory*
- *1 mark: (g) NUMA correctly placed under Shared Memory*
- *1 mark: first valid SMP disadvantage explained (e.g. memory/bus contention limiting scalability)*
- *1 mark: second valid SMP disadvantage explained (e.g. cache-coherence overhead / synchronization cost)*
- *1 mark: clarity/accuracy of terminology across the answer (or split the two disadvantages as 1.5 marks each if the examiner prefers)*

Model confidence: high.

Model note: Figure 4 was supplied only as a caption/crop description; the label letters (a)-(g) and their positions are inferred from the standard Stallings taxonomy of parallel processor architecture. If the actual figure numbers the boxes differently, map the same content labels to the corresponding letters. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 7 (10 marks)

Source: CSC429-2014-12, page [3]

© Hak Cipta Universiti Teknologi MARA

Unofficial model answer:

Registers are small, high-speed storage locations inside the CPU used for temporary data and control information. Five common types are: (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is automatically incremented after each fetch and can be loaded by branch/jump instructions. (2) Instruction Register (IR): holds the instruction currently being decoded and executed; its contents are split into opcode and operand fields by the control unit. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed by the next read or write operation; it connects the CPU to the address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data read from or to be written to memory; it connects the CPU to the data bus and acts as a buffer between the CPU and main memory. (5) Accumulator (AC): a general-purpose data register that holds one operand and the result of arithmetic/logic operations performed by the ALU. Other acceptable examples include general-purpose registers (GPRs), status/flag register (PSW), stack pointer (SP), and index registers.

Marking points:

• 1 mark: Program Counter (PC) — holds address of next instruction; incremented after fetch; loaded by branch instructions.

• 1 mark: Instruction Register (IR) — holds current instruction during decode/execute; opcode and operand fields.

• 1 mark: Memory Address Register (MAR) — holds address of memory location for read/write; drives address bus.

• 1 mark: Memory Data Register (MDR/MBR) — holds data read from or written to memory; buffers data bus.

• 1 mark: Accumulator (AC) — holds operand and result of ALU arithmetic/logic operations.

• 1 mark: Any other valid register (GPR, status/flag/PSW, stack pointer, index register) correctly described, up to the required five.

Model confidence: high.

Model note: No figure or table was supplied; answer is based on standard computer-organization register definitions. The question asks for FIVE registers, so the marking allocates 2 marks per register (description + function) for 10 marks total; the list above gives one mark per register and one mark for a correct function, which is a common alternative allocation. Other valid registers may be accepted if correctly described.

QUESTION 8 (10 marks)

Source: CSC429-2026-02, page [3]

Unofficial model answer:

a) Convert -27.15625$_{10}$ to IEEE-754 single precision (32-bit) hexadecimal. Workflow: (1) Sign: the number is negative, so S = 1. (2) Magnitude: 27.15625$_{10}$. Integer part 27 = 11011$_2$. Fractional part 0.15625 × 2 = 0.3125 → 0; 0.3125 × 2 = 0.625 → 0; 0.625 × 2 = 1.25 → 1; 0.25 × 2 = 0.5 → 0; 0.5 × 2 = 1.0 → 1; so 0.15625 = 0.00101$_2$. Thus 27.15625 = 11011.00101$_2$. (3) Normalize: 11011.00101$_2$ = 1.101100101$_2$ × 2⁴, so exponent E = 4 and mantissa fraction = 101100101. (4) Biased exponent: 4 + 127 = 131 = 10000011$_2$. (5) Assemble: S=1, exponent=10000011, fraction=10110010100000000000000 (23 bits, pad with zeros). Bit string: 1 10000011 10110010100000000000000. Group into nibbles: 1100 0001 1101 1001 0100 0000 0000 0000 = C1D94000$_{16}$. Answer: 0xC1D94000. b) Convert 0x41E18000 to decimal. Workflow: (1) Expand hex to binary: 4=0100, 1=0001, E=1110, 1=0001, 8=1000, 0=0000, 0=0000, 0=0000 → 0100 0001 1110 0001 1000 0000 0000 0000. (2) Split fields: S = 0 (positive); biased exponent = 10000011$_2$ = 131; fraction = 11000011000000000000000$_2$. (3) Unbias: E = 131 − 127 = 4. (4) Significand: 1.fraction = 1.11000011$_2$. (5) Value = 1.11000011$_2$ × 2⁴ = 11100.0011$_2$. (6) Convert: integer 11100$_2$ = 28; fraction 0.0011$_2$ = 1/8 + 1/16 = 0.1875. Total = 28.1875. Answer: +28.1875$_{10}$.

Marking points:

• a) 1 mark: correct sign bit S = 1 for the negative number.

• a) 1 mark: correct binary conversion of integer part 27 = 11011$_2$.

• a) 1 mark: correct binary conversion of fractional part 0.15625 = 0.00101$_2$ (or equivalent correct repeated-multiplication workflow).

© Hak Cipta Universiti Teknologi MARA

• a) 1 mark: correct normalization $1.101100101_2 \times 2^4$ and identification of exponent 4 and 23-bit fraction.

• a) 1 mark: correct biased exponent 131 = 10000011_2.

• a) 1 mark: correct final 32-bit field assembly and hexadecimal result 0xC1D94000 (accept C1D94000h).

• b) 1 mark: correct hex-to-binary expansion of 0x41E18000.

• b) 1 mark: correct field split: S = 0, biased exponent = 131, fraction = 11000011000000000000000.

• b) 1 mark: correct unbiased exponent E = 4 and significand 1.11000011_2.

• b) 1 mark: correct final decimal value +28.1875 (accept 28.1875).

Model confidence: high.

Model note: The question text contains OCR/typographical corruption: '-27.1562510' is interpreted as -27.15625_{10} and '4'l E18O00n' as 0x41E18000. The answer assumes standard IEEE-754 single precision with 1 sign bit, 8 biased exponent bits (bias 127), and 23 fraction bits. No figure or table was supplied.

QUESTION 9 (10 marks)

Source: CSC429-2015-06, page [4]

Unofficial model answer:

a) An interrupt is a signal (hardware or software) sent to the CPU that temporarily suspends the currently executing program and transfers control to a special routine called an interrupt service routine (ISR) or interrupt handler. The CPU saves the current context (program counter and status), executes the ISR to service the interrupting device or event, and then restores the context to resume the interrupted program. Interrupts allow the CPU to respond to asynchronous events without polling.

b) Two different types of interrupt: (1) Hardware interrupt (also called external interrupt) — generated by an external device or hardware signal, e.g., a keyboard key press, a timer expiry, or an I/O device requesting service. It is asynchronous to the running program and is delivered via an interrupt request line to the CPU. (2) Software interrupt (also called internal interrupt or trap) — generated by the execution of an instruction or by an exceptional condition within the CPU, e.g., a system call instruction, division by zero, or an invalid opcode. It is synchronous with the program and is often used to request operating-system services. (Other acceptable pairs: maskable vs non-maskable, vectored vs non-vectored, edge-triggered vs level-triggered.)

c) Advantage: Interrupts improve CPU utilization and system responsiveness because the CPU does not need to waste time polling devices; it can execute useful work and is notified only when service is required, allowing efficient handling of asynchronous events and enabling multitasking. Disadvantage: Interrupts introduce overhead and complexity — each interrupt requires context saving/restoring and ISR execution, which consumes CPU cycles; frequent interrupts can degrade performance, and the system must carefully manage priorities, nesting, and race conditions, increasing design complexity and potential for errors.

Marking points:

• 1 mark: Correct definition of interrupt as a signal that suspends current program and transfers control to an ISR (must mention CPU/context switch or handler).

• 1 mark: Additional detail in definition (e.g., saving/restoring context, asynchronous event, or purpose of responding to events).

• 2 marks: First type of interrupt correctly named and described (e.g., hardware/external with example).

• 2 marks: Second type of interrupt correctly named and described (e.g., software/internal/trap with example).

• 2 marks: One valid advantage clearly explained (e.g., improved CPU utilization, no polling, responsiveness).

• 2 marks: One valid disadvantage clearly explained (e.g., overhead, complexity, priority management).

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify a particular taxonomy of interrupts; the answer uses the common hardware/software classification, but other valid classifications (maskable/non-maskable, vectored/non-vectored) would also be acceptable. Mark allocation is inferred from the 10-mark total and subpart structure (a: 2, b: 4, c: 4).

© Hak Cipta Universiti Teknologi MARA

QUESTION 10 (10 marks)

Source: CSC429-2025-02, page [3]

Unofficial model answer:

Part (a) — Concept identification (4 marks). The four scenarios map onto the classic *architecture vs. organization* distinction. (i) "Deals with the logical aspects of the system, including instruction sets, data formats, and addressing modes" → **Computer Architecture** (the programmer-visible, logical view). (ii) "Defines the functionality, capabilities, and programming model" → **Computer Architecture** (it specifies *what* the machine can do and how it is programmed). (iii) "Concerned with how the computer will execute the tasks and implement the functions" → **Computer Organization** (it specifies *how* the architectural functions are realised). (iv) "Involves the hardware layout, circuits, control signals, memory access, and timing" → **Computer Organization** (the physical/implementation level). In short: architecture = *what* the system does (logical, programmer-visible); organization = *how* it does it (physical, implementation-dependent).

Part (b) — Three CPU registers used in data processing (6 marks). (1) **Accumulator (AC)** — a general-purpose data register that holds one operand before an arithmetic/logic operation and receives the result (e.g., ADD loads the operand and stores the sum), so it is central to ALU data processing. (2) **General-purpose registers (GPRs)** — a small set of fast registers (e.g., R0–R15) that hold operands, intermediate results and data being manipulated by arithmetic/logic instructions, reducing memory traffic. (3) **Status/Flag register (Program Status Word, PSW)** — records the outcome/condition of the last arithmetic operation (zero, carry, sign, overflow flags) which is used for conditional branching and to reflect the result of processing. (Acceptable alternatives that also participate in data processing: **Data Register (DR/MDR)** buffering data to/from the ALU and memory, and **Temporary registers** inside the ALU holding operands during an operation.) Each register should be named and its role in arithmetic/data processing explained.

Marking points:

- *Part (a): 1 mark each for the four rows — (i) Computer Architecture, (ii) Computer Architecture, (iii) Computer Organization, (iv) Computer Organization (4 marks total).*
- *Part (b): 1 mark for correctly naming each of three registers (3 marks).*
- *Part (b): 1 mark for a correct description of each register's role in data/arithmetic processing (3 marks).*
- *Part (b) total 6 marks; award partial credit where a register is named but its function is vague or incorrect.*
- *Overall: 4 + 6 = 10 marks.*

Model confidence: high.

Model note: The scenario table is supplied only as extracted text (no figure); the mapping to architecture/organization is inferred from standard CSC429 definitions (architecture = logical/programmer-visible; organization = physical/implementation). Part (b) accepts any three valid data-processing registers; the model answer lists the most common ones plus acceptable alternatives.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2025-07, page [5]

Unofficial model answer:

The equation in Figure 1 is read as z = (A² − B*C + jl_)/(D + 5). The OCR text is garbled ('jl_' and 'D+5' appear on a separate line), so the expression is interpreted as a numerator (A² − B*C + jl_) divided by the denominator (D + 5). Because the instruction set in Table 1 contains only ADD, SUB, MUL and DIV (no square or jump instructions), A² must be computed as A*A, and the term jl_ is treated as a single named operand/variable (call it J). Registers A, B, C, D are fixed; x, y, z are temporary, and the final result must end in z. The computation order is: t1 = A*A; t2 = B*C; t3 = t1 − t2; t4 = t3 + J; t5 = D + 5; z = t4 / t5.

(a) THREE-ADDRESS form. Each instruction names two sources and one destination, so no explicit temporaries in memory are needed beyond the named registers:

```
MUL (A, A, x)     ; x = A*A
MUL (B, C, y)     ; y = B*C
SUB (x, y, x)     ; x = A*A - B*C
ADD (x, J, x)     ; x = A*A - B*C + J
ADD (D, 5, y)     ; y = D + 5
DIV (x, y, z)     ; z = (A*A - B*C + J)/(D + 5)
```

This is the most compact form: six instructions, no PUSH/POP traffic.

(b) ONE-ADDRESS form. There is a single implicit accumulator; LOAD/STOR move between accumulator and memory, and ADD p/SUB p/MUL p/DIV p operate accumulator ← accumulator op p. Temporaries x and y hold intermediate values:

```
LOAD A        ; ACC = A
MUL A         ; ACC = A*A
STOR x        ; x = A*A
LOAD B        ; ACC = B
MUL C         ; ACC = B*C
STOR y        ; y = B*C
LOAD x        ; ACC = A*A
SUB y         ; ACC = A*A - B*C
ADD J         ; ACC = A*A - B*C + J
STOR x        ; x = numerator
LOAD D        ; ACC = D
ADD #5        ; ACC = D + 5   (immediate 5; if only memory operands allowed, STOR a constant 5 first)
STOR y        ; y = D + 5
LOAD x        ; ACC = numerator
DIV y         ; ACC = numerator/(D + 5)
STOR z        ; z = final answer
```

(c) ZERO-ADDRESS form. A stack machine: PUSH p pushes operand p, POP p pops top of stack into p, and ADD/SUB/MUL/DIV pop two operands and push the result (second-popped op top-popped). Postfix evaluation of (A*A − B*C + J)/(D + 5):

```
PUSH A
PUSH A
MUL           ; stack: A*A
PUSH B
PUSH C
```

© Hak Cipta Universiti Teknologi MARA

```
MUL         ; stack: A*A, B*C
SUB         ; stack: A*A - B*C
PUSH J
ADD         ; stack: A*A - B*C + J
PUSH D
PUSH 5
ADD         ; stack: numerator, D+5
DIV         ; stack: numerator/(D+5)
POP z       ; z = final answer
```

In all three versions the final result is stored in z, as required, and only the permitted instructions from Table 1 are used.

Marking points:

*• 2 marks: correct interpretation of Figure 1 as z = (A*A - B*C + J)/(D + 5), including handling A^2 as A*A and treating the garbled 'jl_' term as a single operand J (award partial if the denominator grouping is missed).*

• 5 marks: three-address program — correct use of MUL/SUB/ADD/DIV with (p,q,r) operand order, correct evaluation order, temporaries x and y used legally, final DIV writing to z (about 1 mark per correct instruction grouping).

• 6 marks: one-address program — correct LOAD/STOR/ADD p/SUB p/MUL p/DIV p sequence, accumulator used correctly, temporaries x and y for numerator and denominator, final STOR z (deduct for missing STOR or wrong operand order).

• 6 marks: zero-address program — correct postfix ordering with PUSH/POP/ADD/SUB/MUL/DIV, stack discipline correct (two pops, one push per arithmetic op), final POP z.

• 1 mark: overall correctness — all three programs compute the same expression and leave the answer in z, using only instructions listed in Table 1.

Model confidence: medium.

*Model note: Figure 1 is supplied only as a cropped image with garbled OCR text ('z=A2-B*C+jl_' and 'D+5'); the exact numerator term (appearing as 'jl_') and the division grouping are inferred. If the intended expression differs (e.g. the denominator applies only to part of the numerator, or 'jl_' is two operands j and l), the instruction sequences must be adjusted accordingly. The one-address immediate 'ADD #5' assumes an immediate form; if only memory operands are permitted, a constant 5 must be stored to a temporary first. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA